AgroNex-Pi

Autonomous Smart Agricultural Robot

Full Project Technical Report & Developer Manual

Raspberry Pi 4 | TensorFlow Lite | AWS Cloud | GPS Navigation

Soil Moisture Sensing | Plant Disease AI | Grass Cutting | Irrigation

Prepared by: **Nikhil & Team**

Academic Year: 2025-2026

Major Project — Final Year B.E./B.Tech (Electronics / Computer Engineering)

Table of Contents

§	Section	Pg.
1.	Project Overview & Objectives	3
2.	System Architecture	4
3.	Hardware Components & Bill of Materials	5
4.	Pin Configuration & Wiring Guide	7
5.	Software Modules — Design & Code Explanation	10
5.1	config.py — Global Configuration	10
5.2	hardware.py — Hardware Driver Layer	11
5.3	irrigation.py — Irrigation Controller	13
5.4	cutter.py — Grass Trimmer Controller	14
5.5	navigation.py — GPS Waypoint Navigation	15
5.6	disease_detect.py — AI Disease Detection Pipeline	16
5.7	disease_db.py — Disease Knowledge Base	18
5.8	main.py — System Supervisor & FSM	19
5.9	app.py — Flask REST API Server	21
5.10	aws_relay.py — AWS Cloud Integration	22
5.11	database.py — SQLite Persistence Layer	23
6.	AI Model Training Pipeline	24
7.	AWS Cloud Architecture	26
8.	Web Dashboard (index.html)	27
9.	System Working — End-to-End Flow	28
10.	Safety Mechanisms & Interlocks	30
11.	Testing Scripts	31
12.	Known Issues & Future Work	32

1. Project Overview & Objectives

AgroNex-Pi is a fully autonomous Raspberry Pi-based smart agricultural robot integrating five subsystems on a single rover chassis: real-time soil moisture sensing & automated irrigation, scheduled grass trimming, GPS-guided waypoint navigation, AI-powered plant disease detection, and live cloud telemetry via AWS. All subsystems are coordinated by a multi-threaded Python supervisor running a Finite State Machine (FSM).

1.1 Key Objectives

- Automate repetitive agricultural tasks (watering, mowing) without continuous human intervention.
- Detect plant diseases early using edge AI (TensorFlow Lite on Raspberry Pi 4) to minimize crop loss.
- Provide real-time sensor telemetry accessible from anywhere via an AWS-hosted web dashboard.
- Operate autonomously within a defined GPS-bounded area using compass-guided navigation.
- Implement fail-safe hardware interlocks to prevent equipment damage during simultaneous actuator operation.
- Support over-the-air monitoring and remote control via Flask REST API and AWS API Gateway.
- Log all events (irrigation cycles, disease scans, system errors) to a local SQLite database for analysis.

1.2 Project Scope

The project targets Tulsi (Holy Basil) and general leafy plants as primary disease detection subjects, with a dual-model architecture supporting both the custom Tulsi model (5 classes: Healthy, Infected, Infested, Wilting, Background) and the PlantVillage broad-crop model (Tomato, Potato, Pepper — 8 classes). The rover is intended for outdoor garden/farm deployment with a human safety observer present.

2. System Architecture

AgroNex-Pi uses a layered, modular software architecture. No module other than hardware.py accesses GPIO or SPI pins directly. All inter-module communication is via Python object references or the shared telemetry cache maintained by app.py.

2.1 Software Layer Diagram

Layer	Modules	Responsibility
Hardware Abstraction (HAL)	hardware.py	GPIO, SPI, I2C, Camera, PWM — all physical I/O
Sensor/Actuator Controllers	irrigation.py cutter.py navigation.py disease_detect.py	Business logic for each subsystem (hysteresis, scheduling, AI inference)
Persistence	database.py	SQLite: events, scans, irrigation logs
Application Layer	app.py (Flask)	REST API, telemetry cache
System Supervisor	main.py	FSM, thread lifecycle, signal handlers
Cloud Integration	aws_relay.py	HTTP push to AWS API Gateway / S3
Knowledge Base	disease_db.py	Plant disease encyclopedia (cause, treatment)
Configuration	config.py	Centralised constants and pin map

2.2 Thread Model

The supervisor launches four concurrent threads managed by Python's threading library:

Control Loop Thread: 3-second poll cycle: reads all sensors, evaluates irrigation, cutter, navigation, triggers auto-scan. Heartbeat timestamp updated each iteration.

Disease Scanner Thread: Runs DiseaseDetector when triggered by green-pixel detection or manual API request. Saves result + JPEG scan to disk.

Flask API Server Thread: Serves the REST API on port 5000 (daemon thread). Provides /api/telemetry, /api/scan, /api/manual/* endpoints.

Watchdog Thread: Monitors control loop heartbeat. If no update within 30 seconds, logs an alert and optionally triggers safe-state.

3. Hardware Components & Bill of Materials

#	Component	Model / Spec	Qty	Purpose
1	Microcontroller	Raspberry Pi 4 Model B (4GB RAM)	1	Central processing unit; runs all Python modules
2	Soil Moisture Sensor	Capacitive v1.2 (3.3V)	1	Reads soil moisture (analog output)
3	ADC (I2C)	ADS1115 16-bit 4-ch I2C	1	Converts analog sensor signals to digital (primary)
4	ADC (SPI Fallback)	MCP3008 10-bit 8-ch SPI	1 (opt)	Backup ADC via SPI bus
5	Water Level Sensor	HW-038 3-pin Analog	1	Monitors water reservoir level
6	Temp/Humidity Sensor	DHT11 or DHT22	1	Ambient temperature and humidity
7	GPS Module	NEO-7M UART @ 9600 baud	1	Global positioning for waypoint navigation
8	Compass/Magnetometer	HMC5883L I2C 3-axis	1	Heading feedback for navigation
9	Camera	Raspberry Pi Camera v2 / USB webcam	1	Captures leaf images for disease AI
10	Motor Driver	BTS7960 43A H-Bridge	2	Drives DC drive motors (left side + right side)
11	Drive Motors	12V DC Geared Motor	4	Chassis locomotion
12	Relay Module	2-Channel 5V Active LOW	1	Switches water pump (CH1) and cutter (CH2)
13	Water Pump	Mini Submersible 5V-12V DC	1	Irrigation water delivery
14	Grass Cutter Motor	12V DC Brushed (blade attached)	1	Grass trimming actuator
15	Buzzer	5V Active Buzzer	1	Audio alerts
16	LED Indicator	Red LED + 220Ohm resistor	1	Visual status indicator
17	Buck Converter	LM2596 / XL4016 DC-DC Step-down	1	Steps 12V battery to 5V for Pi
18	Battery	12V 7Ah SLA or 3S Li-ion Pack	1	Main power supply
19	MicroSD Card	32GB+ Class 10 / A1	1	OS + project files storage

4. Pin Configuration & Wiring Guide

All GPIO assignments use BCM numbering (GPIO.setmode(GPIO.BCM)). Physical pin numbers refer to the standard Raspberry Pi 40-pin header.

4.1 Power & Ground Rail

Physical Pin	Signal	Connected To
Pin 1 (3.3V)	3.3V Power	DHT VCC, ADS1115 VDD, MCP3008 VDD/VREF, HMC5883L VCC, Capacitive Sensor VCC
Pin 2 (5V)	5V Power	Relay Module VCC, Buzzer (+)
Pin 4 (5V)	5V Power	Buck Converter Output to Pi GPIO 5V rail
Pin 6 (GND)	Signal GND	DHT GND
Pin 25 (GND)	Analog GND	ADS1115 GND, MCP3008 AGND (pin 14) + DGND (pin 9)
Pin 39 (GND)	Power GND	BTS7960 logic ground (star-grounded with battery negative)

4.2 GPIO Signal Pins (BCM Numbering)

BCM GPIO	Physical Pin	Direction	Signal / Function	Connected Hardware
GPIO 4	Pin 7	INPUT	DHT Data (10k pull-up to 3.3V)	DHT11/22 Temperature & Humidity Sensor
GPIO 17	Pin 11	OUTPUT	Relay CH1 Pump (Active LOW)	Relay Module IN1 to Water Pump
GPIO 27	Pin 13	OUTPUT	Relay CH2 Cutter (Active LOW)	Relay Module IN2 to Grass Cutter Motor
GPIO 22	Pin 15	OUTPUT	Buzzer (+)	5V Active Buzzer positive terminal
GPIO 23	Pin 16	OUTPUT	LED Anode (via 220 Ohm)	Red Status LED
GPIO 5	Pin 29	PWM OUT	Left Side RPWM (Forward)	BTS7960 #1 RPWM
GPIO 6	Pin 31	PWM OUT	Left Side LPWM (Reverse)	BTS7960 #1 LPWM
GPIO 13	Pin 33	PWM OUT	Right Side RPWM (Forward)	BTS7960 #2 RPWM
GPIO 19	Pin 35	PWM OUT	Right Side LPWM (Reverse)	BTS7960 #2 LPWM
GPIO 14	Pin 8	UART TX	GPS UART TX (optional config)	NEO-7M RX (optional)
GPIO 15	Pin 10	UART RX	GPS UART RX /dev/serial0	NEO-7M TX to Pi RXD

4.3 I2C Bus (GPIO2=SDA, GPIO3=SCL — Physical Pins 3, 5)

I2C Address	Device	Function	Notes
0x48	ADS1115 16-bit ADC	Reads: Soil Moisture (A0), Water Level (A1), Spare (A2/A3)	Internal pull-up enabled; external 4.7k optional
0x1E	HMC5883L Magnetometer	3-axis compass heading for GPS navigation	External 4.7k pull-ups to 3.3V required on SDA/SCL

4.4 SPI Bus (MCP3008 Fallback ADC)

BCM GPIO	Physical Pin	SPI Signal	MCP3008 Pin
GPIO 10	Pin 19	MOSI (DIN)	Pin 11
GPIO 9	Pin 21	MISO (DOUT)	Pin 12
GPIO 11	Pin 23	SCLK (CLK)	Pin 13
GPIO 8	Pin 24	CE0 (CS/SHDN)	Pin 10
3.3V	Pin 1	VDD & VREF	Pins 16 & 15
GND	Pin 25	AGND & DGND	Pins 14 & 9

4.5 BTS7960 Motor Driver Wiring

Two BTS7960 H-Bridge boards are used. BTS #1 drives both left-side motors (front-left + rear-left in parallel). BTS #2 drives both right-side motors. Each board's R_EN and L_EN must be tied to 5V (always enabled). All grounds must share a common star-ground point.

BTS Board	Pi BCM	Pi Physical Pin	BTS Pin	Motor Direction
BTS #1 (Left)	GPIO 5	Pin 29	RPWM	Left Forward
BTS #1 (Left)	GPIO 6	Pin 31	LPWM	Left Reverse
BTS #2 (Right)	GPIO 13	Pin 33	RPWM	Right Forward
BTS #2 (Right)	GPIO 19	Pin 35	LPWM	Right Reverse

WARNING: Always connect a common ground between the Pi, BTS7960 boards, and battery negative terminal. Floating grounds will cause erratic motor behavior or GPIO damage.

4.6 Camera Connection

The Raspberry Pi Camera v2 connects to the CSI ribbon cable port on the Pi 4. A USB webcam is automatically detected as fallback via cv2.VideoCapture() if the CSI camera is absent. The camera is managed as a singleton in hardware.py using the Camera.get_instance() pattern to prevent resource locks.

5. Software Modules — Design & Code Explanation

5.1 config.py — Global Configuration & Constants

Purpose: Single source of truth for all system constants. Every parameter — pin numbers, thresholds, file paths, timing intervals — is defined here. No other module stores configuration values locally.

Constant / Group	Value / Range	Explanation
PIN_DHT_DATA	GPIO 4	Data pin for DHT11/22 sensor
PIN_RELAY_PUMP	GPIO 17	Active-LOW relay for water pump
PIN_RELAY_CUTTER	GPIO 27	Active-LOW relay for grass cutter
PIN_BUZZER / PIN_LED	GPIO 22, 23	Buzzer and LED indicator pins
PIN_LEFT_RPWM/LPWM	GPIO 5, 6	BTS7960 #1 PWM pins (left motors)
PIN_RIGHT_RPWM/LPWM	GPIO 13, 19	BTS7960 #2 PWM pins (right motors)
MOISTURE_LOW / HIGH	35% / 65%	Hysteresis band for irrigation control
PUMP_MAX_RUNTIME	60 seconds	Safety cutoff for stuck pump / dry tank
PUMP_COOLDOWN	120 seconds	Minimum off-time between irrigation cycles
CUTTER_INTERVAL	3600 seconds	1 hour between automatic grass trim runs
CUTTER_DURATION	30 seconds	Duration of each trim cycle
CONFIDENCE_THRESHOLD	0.70 (70%)	Minimum AI confidence to report disease detection
LAPLACIAN_VAR_MIN	12.0	Blur detection threshold (Laplacian variance)
BRIGHTNESS_MIN/MAX	15 / 250	Luminance range for valid leaf images
GREEN_PIXEL_THRESHOLD	0.15 (15%)	Min green pixels in frame to trigger auto-scan
ACTIVE_MODEL_PROFILE	'custom' or 'plantvillage'	Selects which TFLite model to load at startup
AWS_API_URL	API Gateway URL	AWS endpoint for telemetry push
AWS_RELAY_INTERVAL	5 seconds	Interval between cloud telemetry pushes

The load_runtime_config() function reads runtime_config.json at startup, allowing moisture thresholds saved from the dashboard to persist across reboots.

5.2 hardware.py — Hardware Driver Layer (HAL)

Purpose: The Hardware Abstraction Layer. This is the ONLY module that touches GPIO, SPI, I2C, or physical device registers. All other modules receive hardware objects via dependency injection.

Key Classes & Functions:

initialize_hardware_libs(simulate): Imports all hardware libraries (RPi.GPIO, picamera2, adafruit_dht, smbus2, cv2). If simulate=False and RPi.GPIO is absent, raises RuntimeError — no silent fallbacks.

ADS1115ADC: 16-bit I2C ADC driver using raw smbus2 register writes. Reads 4 analog channels. Used for soil moisture (A0) and water level (A1).

MCP3008ADC: 10-bit SPI ADC fallback driver using spidev. Activated if ADS1115 is not configured.

SoilSensor: Wraps ADS1115 Channel 0. Calibrates raw 16-bit counts to 0-100% moisture. Rejects implausible readings.

WaterLevelSensor: Wraps ADS1115 Channel 1. Converts raw counts to reservoir fill percentage (0-100%).

DHTSensor: Reads temperature and humidity from DHT11/22 via adafruit-circuitpython-dht. Returns last valid reading on CRC error.

GPSSensor: Reads NMEA sentences from NEO-7M GPS via PySerial on /dev/serial0. Parses $GPRMC for lat/lon/speed. Returns fix age for staleness rejection.

CompassSensor: Reads HMC5883L magnetometer via I2C. Computes heading in degrees (0-360) with optional declination correction.

Relay: Controls 2-channel 5V relay module via GPIO (Active LOW). pump_on(), pump_off(), cutter_on(), cutter_off() with safety interlock preventing simultaneous operation.

Motors: Controls 4 drive motors via 2x BTS7960 using software PWM at 1kHz. forward(), reverse(), turn_left(), turn_right(), stop().

Camera: Singleton camera manager. Supports CSI (picamera2) with USB webcam fallback. Exposes capture_frame() returning numpy BGR array.

Alerts: Manages buzzer and LED. alert_disease() triggers beep + LED flash. alert_water_low() triggers sustained buzzer.

Simulation Mode: When --simulate CLI flag is passed, all hardware classes return realistic mock data (random moisture %, simulated GPS coordinates). A persistent SIMULATION MODE warning banner appears in logs and dashboard.

5.3 irrigation.py — Irrigation Controller

Design Pattern: Hysteresis-based on/off control with safety interlocks. Called every 3 seconds from the Control Loop.

Step 1 — Read Sensor: Calls SoilSensor.read_percentage(). If moisture_pct is None (implausible ADC reading), cycle is skipped and event logged.

Step 2 — Active Irrigation Check: If pump is ON: checks if moisture >= 65% (normal stop) or runtime > 60s (safety timeout stop).

Step 3 — Activation Decision: If pump OFF and moisture < 35%: checks cooldown >= 120s since last off then starts pump. Before starting, calls motors.stop() to halt chassis.

Step 4 — Normal State: If moisture 35%-65%: status = MOISTURE_OK, no action.

All irrigation cycle events (start time, end time, duration, initial moisture, final moisture, trigger type) are logged to the SQLite database for efficiency analysis.

5.4 cutter.py — Grass Trimmer Scheduler

Schedules automated grass trimming every CUTTER_INTERVAL (3600s = 1 hour) for CUTTER_DURATION (30s). Before energizing the cutter relay, chassis motors are hard-stopped to prevent voltage sag on the shared 12V bus.

Safety Interlocks:

- Refuses to start if CUTTER_ENABLED = False in config.py.
- Refuses to start if water pump is currently active (prevents simultaneous high-current loads).
- Re-enforces motors.stop() on every control loop tick while cutting is active.
- Emergency stop method immediately de-energizes relay with no delay.

5.5 navigation.py — GPS Waypoint Navigation

Implements GPS + compass guided autonomous navigation. The rover moves in short, safe bursts (NAV_DRIVE_SLICE = 1.0s, NAV_TURN_SLICE = 0.2s) and stops after each slice to re-check GPS and heading.

Navigation Algorithm (execute_waypoint_step):

1. Stop motors first (safety).
2. Request fresh GPS fix; reject if fix age > 5 seconds.
3. Compute great-circle distance and bearing to current waypoint using Haversine formula.
4. If distance <= 2.5m (NAV_WAYPOINT_REACHED_M) mark waypoint reached, advance index.
5. Compute heading error = bearing minus current_heading (wrapped to -180 to +180 degrees).
6. If heading error > 12 degrees (NAV_HEADING_TOLERANCE_DEG) pivot left/right for 0.2s then stop.
7. If heading error <= 12 degrees drive forward for 1.0s then stop.
8. Return status dict (action, distance, bearing, heading_error).

5.6 disease_detect.py — AI Plant Disease Detection Pipeline

Runs TensorFlow Lite inference on captured leaf images. Supports three TFLite runtime backends: tflite_runtime (legacy), ai_edge_litert (new Raspberry Pi OS package), and tensorflow.lite (full TensorFlow fallback).

Inference Pipeline Steps:

Image Quality Gate: Checks Laplacian variance (blur < 12.0) and mean luminance (outside 15-250 range). Rejects bad frames with RETAKE_NEEDED flag — NOT a disease alert.

Green Pixel Verification: HSV-based green pixel count must exceed 15% of frame to confirm a leaf is present before running inference.

Segmentation Consensus: Image divided into 3x3 grid (9 segments). Each classified independently. Disease alert only if diseased segments > 50% (majority voting) reducing false positives.

TFLite Inference: Image resized to 224x224, normalized to float32 [0.0, 1.0]. Interpreter.invoke() runs on XNNPACK delegate.

Confidence Gate: Only softmax confidence >= 70% accepted. Below this threshold: status = LOW_CONFIDENCE.

Result Enrichment: Matched label looked up in disease_db.py for: common name, affected crop, cause, symptoms, treatment.

Alert & Logging: If disease detected: buzzer alert triggered, result JPEG saved to /static/scans/, full result logged to SQLite DB.

TFLite import fallback chain in disease_detect.py lines 27-44:

```
try: import tflite_runtime.interpreter as tflite # legacy Pi except: pass if tflite
is None: try: import ai_edge_litert.interpreter as tflite # new Pi OS except: pass if
tflite is None: try: import tensorflow.lite as tflite # full TF except: tflite = None
```

5.7 disease_db.py — Plant Disease Knowledge Base

A static Python dictionary containing disease metadata for all model classes: common name, affected crop, pathogen type, cause, symptoms, and recommended treatment.

Model Class Label	Common Name	Pathogen Type	Treatment Summary
Tulsi___Healthy	Tulsi (Healthy)	—	No treatment needed
Tulsi___Infected	Tulsi Leaf Spot / Blight	Fungal (Colletotrichum / Alternaria)	Neem oil spray, remove infected leaves, improve d
Tulsi___Infested	Tulsi Pest Infestation	Insect / Mite	Neem oil, insecticidal soap, manual pest removal
Tulsi___Wilting	Tulsi Wilt / Root Rot	Fungal (Fusarium) / Overwatering	Reduce watering, improve drainage, fungicide dren
background_not_a_leaf	Non-plant background	—	Move camera closer to plant
Tomato___Early_blight	Early Blight	Fungal (Alternaria solani)	Copper-based fungicide, crop rotation
Tomato___Late_blight	Late Blight	Oomycete (Phytophthora infestans)	Systemic fungicide, destroy infected plants
Tomato___healthy	Healthy Tomato	—	—
Potato___Early_blight	Early Blight (Potato)	Fungal (Alternaria solani)	Fungicide spray, remove affected leaves
Potato___Late_blight	Late Blight (Potato)	Oomycete (Phytophthora infestans)	Metalaxyl fungicide, no overhead irrigation
Potato___healthy	Healthy Potato	—	—
Pepper__bell___Bacterial_spot	Bacterial Spot	Bacterial (Xanthomonas)	Copper bactericide, crop rotation
Pepper__bell___healthy	Healthy Bell Pepper	—	—

5.8 main.py — System Supervisor & Finite State Machine

Purpose: Entry point and central coordinator. Launches all threads, runs the FSM control loop, handles OS signals for graceful shutdown.

FSM States:

State	Description	Transitions To
BOOT	System startup — hardware library imports, logging setup	SELF_TEST
SELF_TEST	Verifies GPIO, ADC, camera, TFLite model, database. Enters SAFE_STATE if any test fails.	IDLE (pass) or SAFE_STATE (fail)
IDLE	All sensors active. Awaiting trigger: low moisture, cutter interval, green plant detection, manual API command.	IRRIGATING, CUTTING, NAVIGATING, SCANNING
IRRIGATING	Pump is ON. Chassis is halted. Monitors moisture to reach 65% or timeout.	IDLE
CUTTING	Cutter relay ON. Chassis halted. Times 30s blade cycle.	IDLE
NAVIGATING	GPS+compass guided motion toward next waypoint. One heading correction per GPS fix.	Waypoint reached → IDLE; Obstacle → IDLE; Timeout → IDLE; Emergency → SAFE_STATE
SCANNING	Disease scanner thread active. Captures frame, runs inference, logs result.	IDLE
SAFE_STATE	Emergency or critical failure. All actuators de-energized. Only monitoring active.	IDLE (manual recovery)

Signal Handling:

• SIGTERM (systemd stop): Calls emergency_stop() de-energizes all relays/motors joins all threads exits cleanly.

• SIGINT (Ctrl+C): Same as SIGTERM. No data corruption — SQLite writes are transactional.
• Watchdog: Monitors control loop heartbeat timestamp. If stale > 30s logs WATCHDOG_TIMEOUT alert.

5.9 app.py — Flask REST API Server

Hosts a Flask web server on port 5000. Provides REST endpoints for the dashboard frontend and external integrations. All state is read from/written to a shared telemetry_cache dict (thread-safe with RLock).

Method	Endpoint	Description
GET	/api/telemetry	Returns current telemetry JSON: moisture, temperature, humidity, GPS, heading, R
POST	/api/scan	Triggers an immediate disease scan (background thread). Returns scan ID.
GET	/api/scan/result	Returns latest disease scan result with confidence, label, treatment, image path.
POST	/api/manual/pump/on	Manually starts water pump (with safety checks)
POST	/api/manual/pump/off	Manually stops water pump
POST	/api/manual/cutter/on	Manually starts grass cutter (with safety checks)
POST	/api/manual/cutter/off	Manually stops grass cutter
POST	/api/navigate/waypoint	Adds a GPS waypoint to the navigation mission
POST	/api/navigate/clear	Clears all waypoints and halts navigation
GET	/api/logs	Returns last 100 system log events from SQLite DB
POST	/api/config/thresholds	Updates moisture low/high thresholds at runtime (saved to runtime_config.json)
GET	/api/disease/library	Returns the full disease knowledge base from disease_db.py
GET	/	Serves the main dashboard index.html

5.10 aws_relay.py — AWS Cloud Integration

Background thread that periodically (every 5 seconds) HTTP POSTs telemetry to AWS API Gateway. Also polls the /command endpoint to receive remote control commands from the cloud dashboard.

Pi to AWS Endpoint	Direction	Payload	AWS Service
POST /telemetry	Pi to Cloud	Full sensor JSON	API Gateway to Lambda to S3 (state.json)
GET /command	Pi from Cloud	Command string (e.g. pump_on)	API Gateway to Lambda to S3 (command.json)
POST /scan	Pi to Cloud	JPEG image (base64) + scan result JSON	API Gateway to Lambda to S3 (/scans/)
POST /snapshot	Pi to Cloud	JPEG camera snapshot (base64) every 60s	API Gateway to Lambda to S3 (/snapshots/)

5.11 database.py — SQLite Persistence Layer

Manages a local SQLite database at logs/agronex.db. All writes are thread-safe (threading.Lock). Key tables:

system_events: Logs: timestamp, event_type (FSM_TRANSITION, IRRIGATION_START, CUTTER_START, PUMP_TIMEOUT etc.), severity (INFO/WARNING/ERROR), message, simulated flag.

irrigation_events: Logs each complete pump cycle: start/end ISO timestamps, duration seconds, start moisture %, end moisture %, trigger type.

disease_scans: Logs each disease inference: timestamp, label, common_name, confidence, is_healthy flag, image_path, treatment recommendation.

telemetry_snapshots: Optional periodic snapshots of full sensor telemetry for historical analysis.

6. AI Model Training Pipeline

Two training scripts generate TFLite models for deployment on Raspberry Pi 4.

6.1 train_custom_plant_model.py — Custom Tulsi Trainer

Trains a MobileNetV2-based transfer learning model on a local dataset collected using the Pi camera. Dataset is organized into class subdirectories under field_dataset/.

Dataset Organization: Place images in field_dataset/class_name/ directories. Recommended classes: Tulsi___Healthy, Tulsi___Infected, Tulsi___Infested, Tulsi___Wilting, background_not_a_leaf.

Data Augmentation: RandomFlip, RandomRotation, RandomZoom, RandomBrightness applied on-the-fly via tf.keras.Sequential augmentation pipeline.

Base Model: MobileNetV2 pretrained on ImageNet (include_top=False). Input shape: 224x224x3. Base layers frozen for first 10-15 epochs.

Classification Head: GlobalAveragePooling2D to Dropout(0.4) to Dense(128, ReLU) to Dropout(0.3) to Dense(num_classes, Softmax).

Fine-tuning: After initial training, last 50 layers of MobileNetV2 unfrozen, retrained with lr=1e-5.

Quantization & Export: Model converted to float32 TFLite using TFLiteConverter. Saved as models/custom_plant_model.tflite. Labels saved as models/custom_labels.txt.

```
Training Command: python train_custom_plant_model.py
```

6.2 train_plant_disease_model.py — PlantVillage Broad Crop Trainer

Trains on the PlantVillage dataset (plantdisease.zip) for 8 classes across Tomato, Potato, Bell Pepper. Same MobileNetV2 architecture.

```
Training Command: python train_plant_disease_model.py
```

Both scripts output to models/ directory. Active model is selected by setting AGRONEX_MODEL_PROFILE=custom (default) or AGRONEX_MODEL_PROFILE=plantvillage as an environment variable before launching main.py.

6.3 Model Deployment on Raspberry Pi

1. Train model on Windows/Linux PC (GPU recommended).
2. Copy models/ folder to Raspberry Pi: scp -r models/ nikhil@:~/Major project/models/
3. Verify with: python3 test_disease_camera.py
4. Set model profile if needed: export AGRONEX_MODEL_PROFILE=custom
5. Launch full system: python3 main.py

7. AWS Cloud Architecture

AgroNex-Pi uses a serverless AWS architecture for cloud telemetry, remote control, and scan image storage.

AWS Service	Configuration	Function
API Gateway	REST API, ap-south-1 region, CORS enabled	Entry point for all Pi HTTP POST/GET requests
AWS Lambda	Python 3.11 runtime, ~128MB memory	Processes telemetry payloads, reads/writes S3
Amazon S3	Private bucket: agronex-dashboard	Stores state.json, command.json, scan JPEGs, snapshot JPEGs, static dashbo
S3 Static Hosting	index.html + CSS/JS served from S3	Public-facing dashboard accessible from any browser
CloudWatch Logs	Automatic Lambda log retention	Debugging Lambda function errors

Data Flow:

- Pi to Cloud (Telemetry): Every 5 seconds, aws_relay.py POSTs JSON to /telemetry. Lambda stores it as state.json in S3.
- Cloud to Pi (Command): Every 5 seconds, aws_relay.py GETs /command. Lambda reads command.json from S3. If command exists and unprocessed, Pi executes it.
- Pi to Cloud (Scan Image): After each disease scan, aws_relay.py POSTs the JPEG (base64-encoded) and scan result JSON to /scan.
- Pi to Cloud (Snapshot): Every 60 seconds, a JPEG snapshot from the camera is uploaded to S3 /snapshots/ for live monitoring.
- Cloud Dashboard to S3: The public S3-hosted dashboard fetches state.json every 3 seconds to display live telemetry.

8. Web Dashboard (index.html)

The AgroNex-Pi dashboard is a single-page HTML/CSS/JavaScript application hosted on Amazon S3. It provides real-time telemetry display, manual control buttons, disease scan history, farmer knowledge blog, and live map integration.

Overview Cards: Live tiles for: Soil Moisture %, Water Level %, Temperature, Humidity, GPS coordinates, Heading (animated compass), FSM state, Pump status, Cutter status.

Disease Detection Tab: Latest scan image, detected disease label, confidence bar, treatment recommendation. Retake/Low-Confidence badges on quality gate rejection.

Scan Gallery: Grid of recent scan JPEG thumbnails from S3 /scans/ folder with timestamp and label overlay.

Navigation Tab: Interactive map (Leaflet.js) with current GPS position, waypoint placement, and route visualization.

Disease Library Tab: Full searchable encyclopedia of all supported plant diseases with cause, symptoms, treatment.

Farmer Blogs Tab: Educational articles and farming tips related to crop disease management and autonomous farming.

Logs Tab: Live system event log from /api/logs — FSM transitions, irrigation events, sensor errors.

Manual Controls: Buttons for: Trigger Scan, Pump ON/OFF, Cutter ON/OFF, Set Waypoint, Clear Waypoints.

9. System Working — End-to-End Flow

9.1 Startup Sequence

1. User runs: python3 main.py (or systemd starts it on boot)
2. initialize_hardware_libs() imports all drivers. Fails loudly if Pi hardware is missing without --simulate.
3. GPIO BCM mode set. All relay outputs initialized to OFF (HIGH = deactivated, Active LOW relay).
4. run_self_test() verifies: GPIO, ADC, DHT, GPS, Compass, Camera, TFLite model, SQLite DB.
5. Flask server thread starts on port 5000.
6. AWS Relay thread starts (if AGRONEX_AWS_URL is configured).
7. Watchdog thread starts.
8. Control Loop thread starts (3-second poll cycle).
9. FSM transitions: BOOT to SELF_TEST to IDLE.

9.2 Normal Operation (IDLE / MONITORING Cycle)

Every 3 seconds, the Control Loop performs:

1. Read all sensors: soil moisture, water level, DHT temp/humidity, GPS fix, compass heading.
2. Update telemetry cache (for Flask API and AWS relay).
3. Evaluate Irrigation: If moisture < 35%, start pump cycle. If irrigation active, check completion.
4. Evaluate Cutter: If 3600s have elapsed since last trim, start 30s cutter cycle.
5. Evaluate Navigation: If waypoints are configured, execute one GPS/compass guided motion slice.
6. Evaluate Auto-Scan: If green_pixel_fraction > 15% and cooldown elapsed, trigger disease scanner thread.
7. Push telemetry to AWS (async, via AWSRelay background thread).

9.3 Disease Scan Flow (Auto or Manual)

1. Trigger: auto (green pixel detection) or manual (dashboard button / API POST /api/scan).
2. Camera captures 224x224 frame (Picamera2 CSI or USB webcam fallback).
3. Quality Gate: Laplacian variance check (blur) and luminance check. If rejected: RETAKE_NEEDED (not a false disease alert).
4. Green pixel mask (HSV) applied: confirms leaf is in frame.
5. Image split into 3x3 grid (9 segments). Each segment classified by TFLite model.
6. Majority voting: if diseased_count > 4 (>50%) then DISEASE_DETECTED. Else HEALTHY.
7. Confidence gate: must be >= 70%.
8. Result: label, confidence, common_name, cause, treatment saved to DB + S3.
9. Alert: buzzer + LED if disease detected.

10. Safety Mechanisms & Interlocks

Safety Interlock	Mechanism	Implemented In
Pump Dry-Run Prevention	Checks WATER_CRITICAL_LOW (20%). If tank empty, pump is NOT started even if soil is dry.	irrigation.py
Pump Max Runtime	Pump auto-shuts off after 60s (PUMP_MAX_RUNTIME) regardless of moisture.	irrigation.py
Pump Cooldown	120s minimum between pump cycles (PUMP_COOLDOWN). Protects relay contacts and pump motor.	irrigation.py
Cutter + Pump Interlock	Cutter refuses to start if pump is ON. Pump refuses manual start if cutter is ON.	cutter.py + irrigation.py
Chassis Stop During Actuation	motors.stop() called before starting pump or cutter. Chassis only moves after actuation completes.	irrigation.py, cutter.py
False-Positive Suppression	Disease alert only if >50% of image segments detect it (majority voting).	disease_detector.py
Image Quality Gate	Blurry or overexposed frames rejected with RETAKE_NEEDED (not a false positive).	disease_detector.py
Confidence Gate	AI confidence must be >= 70% for any diagnosis to be reported.	disease_detector.py
SIGTERM/SIGINT Handler	Clean shutdown: emergency_stop() all GPIO LOW, threads joined, exit. No partial hardware states.	main.py
Watchdog Timer	If control loop heartbeat stale > 30s logs WATCHDOG_TIMEOUT.	main.py
Simulation Mode Banner	If --simulate used, persistent WARNING banner prevents accidental production use.	hardware_ports.py

11. Testing Scripts

Script	Purpose
test_all_sensors.py	Reads all sensors simultaneously and prints formatted output. Use to verify hardware connect
test_disease_camera.py	Interactive camera tester + disease inference. Shows live camera feed, captures frame on EN
test_motors.py	Tests all 4 motor directions (forward, reverse, left, right) in sequence. Verify wheel direction be
test_relay.py	Cycles pump and cutter relays ON/OFF. Verify relay click sounds and actuator response.
test_dht.py	Reads DHT11/22 temperature and humidity 10 times with 2s intervals. Checks for CRC errors
test_soil.py	Reads soil moisture ADC value repeatedly and computes percentage. Shows raw ADC + calil
test_water.py	Reads HW-038 water level sensor continuously. Shows raw ADC and reservoir fill %.
test_camera.py	Simple camera connectivity test. Captures one frame and saves test_capture.jpg.
test_compass_raw.py	Reads raw HMC5883L X/Y/Z values and computed heading at 1Hz. Use to calibrate compass
test_navigation_sensors.py	Reads GPS + compass together. Computes bearing and distance to a hardcoded test waypoi
test_alert.py	Triggers buzzer and LED alert patterns. Tests audio and visual alert hardware.
verify_runtime_telemetry.py	Connects to Flask API on localhost:5000 and prints full telemetry JSON. Verifies main.py is ru

12. Known Issues & Future Work

12.1 Known Limitations

• No obstacle detection — a human safety observer is mandatory at all times.

• GPS accuracy is plus or minus 2-5m (consumer NEO-7M). Waypoint arrival radius of 2.5m compensates.

• No wheel encoders — indoor navigation without GPS is not possible.

• Disease model is limited to plants it was trained on. Novel diseases will produce incorrect results.

• Simultaneous inference on multiple plants is not supported.

• AWS cloud relay requires internet connectivity (all features remain functional offline via local Flask dashboard).

12.2 Planned Future Enhancements

• Ultrasonic obstacle detection (HC-SR04) with automatic path re-planning.

• YOLO-based real-time object detection for plant identification and disease localization.

• Battery voltage monitoring via ADC spare channels (A2/A3) with low-battery shutdown.

• Multi-plant scheduling: GPS-mapped plant beds with per-plant disease tracking history.

• Bluetooth/Wi-Fi Direct mode for dashboard access without internet (Pi AP mode).

• Rain sensor integration to inhibit irrigation during rainfall.

• Edge TPU (Google Coral) support for 5x faster inference with INT8 quantized models.

• OTA firmware updates via AWS S3 deployment pipeline.